Programación I

Operadores Lógicos: Formulación de Condiciones Complejas

Hasta ahora hemos trabajado con condiciones simples (ej. A > B). Sin embargo, la lógica de programación a menudo requiere evaluar múltiples factores simultáneamente. Los operadores lógicos nos permiten combinar condiciones simples para crear estructuras de decisión mucho más potentes y precisas.

Introducción

Un operador lógico actúa sobre valores de verdad (Verdadero o Falso). El resultado de una operación lógica es, a su vez, un valor booleano.

En C++, estos valores se representan como **true** (1) y **false** (0).

Los tres operadores fundamentales son:

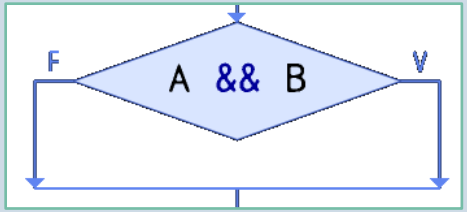
- **AND** (&&)
- **OR** (||)
- **NOT** (!)

Operadores Lógicos

Operador	C++	Significado
AND	**&&**	Exige que todas las proposiciones sean verdaderas para que el resultado de la condición sea verdadero.
OR	**||**	Exige que alguna de las proposiciones sea verdadera para que el resultado de la condición sea verdadero.
NOT	**!**	Invierte el valor lógico de una condición, transformando una proposición verdadera en falsa y una falsa en verdadera.

Operador AND

El operador AND **(&&)** combina dos o más condiciones y devuelve verdadero únicamente cuando todas las condiciones son verdaderas. Si al menos una condición es falsa, el resultado completo es falso.



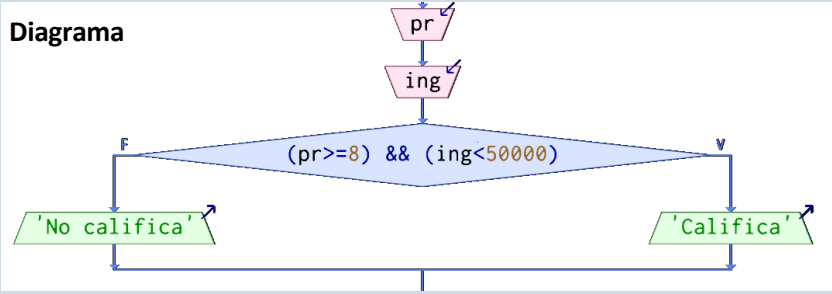
Diagrama

Tabla de verdad

Condición A	Condición B	A && B
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Falso
Falso	Verdadero	Falso
Falso	Falso	Falso

Ejemplo AND

Problema: Verificar si una persona puede obtener una beca. Requisitos: tener promedio mayor o igual a 8 **y** tener ingresos familiares menores a $50.000.



Código C++

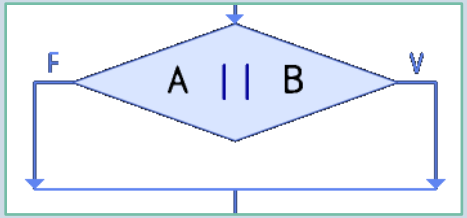
```
float pr, ing;
cin >> pr;
cin >> ing;
if ( (pr >= 8) && (ing < 50000) ) {
        cout << “Califica" << endl;
}
else {
        cout << "No califica" << endl;
}
```

Tabla de verdad

pr >= 8	ing < 50000	pr >= 8 && ing < 5000
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Falso
Falso	Verdadero	Falso
Falso	Falso	Falso

Operador OR

El operador OR (||) combina dos o más condiciones y devuelve verdadero cuando al menos una de las condiciones es verdadera. Solo es falso cuando todas las condiciones son falsas.



Diagrama

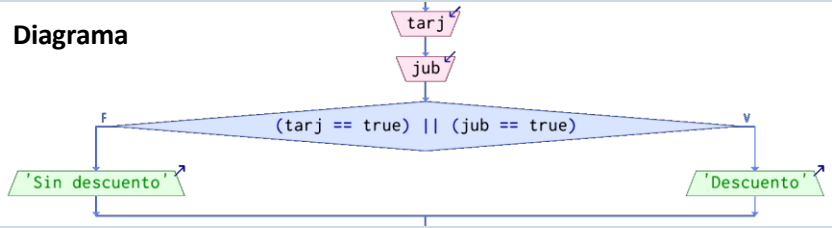
Condición A	Condición B	A || B
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Falso	Verdadero	Verdadero
Falso	Falso	Falso

Tabla de verdad

Ejemplo OR

Problema: Verificar si un producto tiene descuento. Requisitos: Tener tarjeta de socio **o** ser jubilado.

Diagrama



Código C++

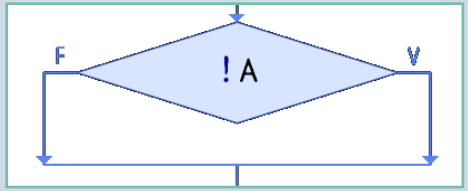
```
bool tarj, jub;
cin >> tarj;
cin >> jub;
if ( (tarj == true) || (jub == true) ) {
   cout << “Descuento" << endl;
}
else {
   cout << “Sin descuento." << endl;
}
```

Tabla de verdad

tarj == true	jub == true	tarj == true || jub == true
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Falso	Verdadero	Verdadero
Falso	Falso	Falso

Operador NOT

El operador NOT (**!**) es un inversor lógico unario. Aplica la **negación**: transforma **true** en **false** y viceversa, es decir, funciona como un interruptor de 'opuestos'; es la herramienta ideal para validar que algo no ha ocurrido o para invertir el resultado de una condición.



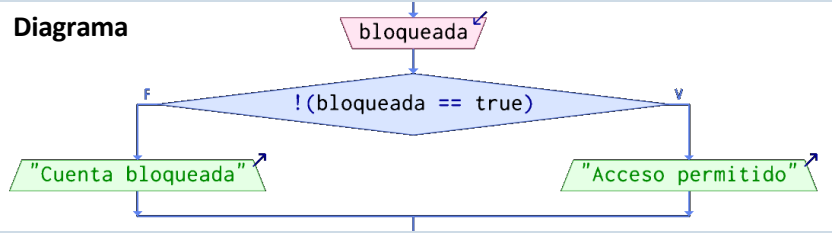
Diagrama

Condición A	NOT A
Verdadero	Falso
Falso	Verdadero

Tabla de verdad

Ejemplo NOT

Problema: Verificar si una cuenta está bloqueada. Requisito: Si la cuenta **NO** está bloqueada, se permite el acceso.



Código C++

```
bool bloqueada;
cin >> bloqueada;
if (!(bloqueada == true)) {
   cout << "Acceso permitido" << endl;
}
else {
   cout << "Cuenta bloqueada<< endl;
}
```

Tabla de verdad

bloqueada == true	! (bloqueada)
Verdadero	Falso
Falso	Verdadero

Prioridad de Operadores Lógicos en C++

El orden de resolución en condiciones complejas
Cuando combinamos múltiples operadores lógicos en una misma instrucción, el compilador no los lee simplemente de izquierda a derecha. Existe una jerarquía interna (precedencia) que determina qué operación se realiza primero.

Tabla de prioridad (de mayor a menor)

Prioridad	Operadores
1	**NOT (!)**
2	**AND (&&)**
3	**OR (||)**

Errores comunes de prioridad

- **Asumir lectura lineal**: Creer que **A || B && C** es lo mismo que **(A || B) && C**. **Error**: Sin paréntesis, el AND se "agrupa" primero.

  Ejemplo: **int A = 1, B = 0, C = 0**;

  ❌ Error común (lectura lineal). Crees que es: **(A || B) && C**

  (1 || 0) && 0 → true && false → **false**

  ✅ Lo que realmente pasa en C++: **A || (B && C)**

  1 || (0 && 0) → true || false → **true**
- **Negaciones mal ubicadas**: **!A && B** solo niega a **A**. Si querés negar el resultado de la conjunción, debés usar **!(A && B)**.
- **Olvidar los Relacionales**: Los operadores relacionales **(>, <, ==)** tienen mayor prioridad que los lógicos.

  Ejemplo: **nota > 7 && nota < 10** se entiende como **(nota > 7) && (nota < 10)** sin necesidad de paréntesis, porque el **>** y **<** se resuelven antes que el **&&**.

  NOTA: Recomendamos siempre usar paréntesis.

Buenas Prácticas

Paréntesis de Claridad: Aunque la jerarquía sea clara para el compilador, usá paréntesis para que sea clara para los humanos.

- Mejor: **(A) || (B && C)**